

连锁餐饮管理系统 — 系统设计文档

项目名称: 宏曦餐饮 (hxfood) 连锁餐饮管理系统文档版本: v1.0 | 创建日期: 2026-07-08

项目负责人: 伍文骏

参与评审: 徐晶 (前端工程师)、李宗浩 (后端工程师)、朱林 (测试验收员)

1. 项目概述

1.1 业务定位

连锁餐饮管理系统，面向快餐/简餐业态（预加工肉类、酱料、半成品），品牌方总部集中管理加盟店，涵盖进销存全链路和加盟全生命周期。

1.2 业务规模

- 量级:** 中大型 (50~200 加盟店, 日均 200~1000 单)
- 品牌:** 多品牌管理, 数据品牌级别隔离
- 扩展:** 2-3 年可扩展至 200+ 门店

1.3 用户角色

角色	入口	核心职责
品牌方总部	Web 管理后台	审核加盟/订单、商品定价、采购管理、全品牌报表
加盟店	微信小程序	浏览商品、提交订单、查订单、管理账户、门店信息
中央厨房	Web 后台	接收生产工单、生产管理、原料/成品库存、发货

供应商	Web/H5 后台	接收采购订单、发货、对账
-----	-----------	--------------

1.4 核心业务流程

加盟入驻：浏览品牌 → 提交申请 → 总部审核 → 缴费 → 开通账号

订货流程：加盟店下单 → 总部审核 → 中央厨房生产 → 发货 → 收货确认

采购流程：总部下采购单 → 供应商发货 → 中央厨房收货 → 质检入库 → 加工生产

结算流程：预充值 / 信用账期 → 订单扣款 → 对账 → 还款 → 开票

2. 技术方案

2.1 技术选型

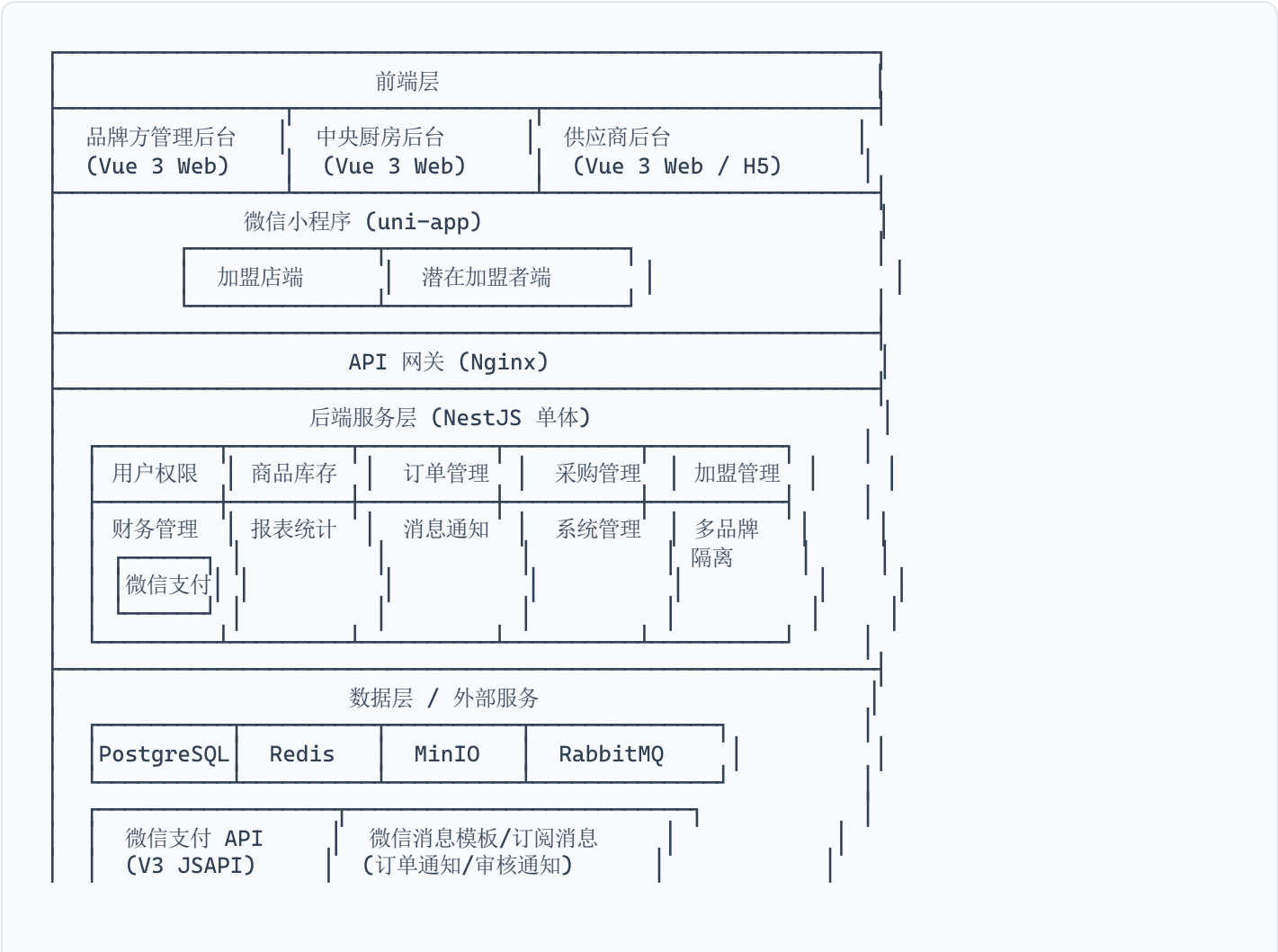
层	技术	版本
后端框架	NestJS + TypeScript	11.x
ORM	Prisma	5.x
数据库	PostgreSQL	16
缓存	Redis	7.x
消息队列	RabbitMQ	3.x
文件存储	MinIO / 阿里云 OSS	—
Web 前端	Vue 3 + shadcn-vue + Pinia + Vite	3.x
小程序	uni-app (Vue 3 模式)	3.x
包管理	pnpm monorepo (Turborepo)	—
CI/CD	GitHub Actions / GitLab CI	—

2.2 选型理由

- **NestJS**: 模块化架构天然适合多角色多模块系统，IoC 容器和装饰器提高开发效率
- **Prisma**: 类型安全的 ORM，对复杂数据模型和关系支持好，Client Extension 支持品牌隔离自动注入
- **PostgreSQL**: 支持 LTREE（商品分类树）、RLS（行级安全）、计算列，多品牌隔离的基石
- **uni-app**: 一套代码编译微信小程序，Vue 3 模式完整支持 Pinia 生态
- **pnpm monorepo**: 前后端共享类型定义、API 层、工具函数，保证接口类型安全

3. 系统架构

3.1 整体架构图



3.2 多品牌数据隔离架构（三层防护）

第一层：应用层

NestJS BrandContextInterceptor

→ 从 JWT + X-Brand-Id 提取 brand_id

→ 注入到 CLS (AsyncLocalStorage)

→ RBAC Guard 校验用户对该品牌的权限

第二层：ORM 层

Prisma Client Extension

→ 自动在 WHERE/CREATE 中注入 brand_id

→ 白名单排除公共表 (Brand, User, Role, Permission)

第三层：数据库层

PostgreSQL Row-Level Security

→ USING (brand_id = current_setting('app.brand_id')::uuid)

→ 应用层 bug 的最后防线

4. 数据库设计

4.1 品牌与组织架构

```
-- 品牌表
brands (
  id          UUID PK DEFAULT gen_random_uuid(),
  name        VARCHAR(100) NOT NULL,
  code        VARCHAR(20) UNIQUE NOT NULL,
  status      ENUM('active','inactive','suspended') DEFAULT 'active',
  config      JSONB,                -- 品牌级配置
  created_at  TIMESTAMPTZ DEFAULT now(),
  updated_at  TIMESTAMPTZ DEFAULT now()
);

-- 统一组织节点（总部/中央厨房/加盟店/供应商/仓库）
organizations (
  id          UUID PK DEFAULT gen_random_uuid(),
```

```

    brand_id      UUID NOT NULL REFERENCES brands(id),
    parent_id     UUID REFERENCES organizations(id),
    org_type      ENUM('headquarters','central_kitchen','franchise_store','supplier','warehouse'),
    name          VARCHAR(200),
    code          VARCHAR(50),
    contact_name  VARCHAR(100),
    contact_phone VARCHAR(20),
    address       JSONB,
    location      GEOGRAPHY(Point),    -- PostGIS 扩展，地图选点和 LBS
    status        ENUM('active','inactive','suspended') DEFAULT 'active',
    config        JSONB,
    created_at    TIMESTAMPTZ DEFAULT now(),
    updated_at    TIMESTAMPTZ DEFAULT now()
);

```

-- 用户表

```

users (
    id          UUID PK DEFAULT gen_random_uuid(),
    openid      VARCHAR(64),          -- 微信小程序 openid
    unionid     VARCHAR(64),          -- 微信开放平台 unionid
    username    VARCHAR(50),
    password_hash VARCHAR(255),
    real_name   VARCHAR(50),
    phone       VARCHAR(20),
    avatar      VARCHAR(500),
    status      ENUM('active','disabled') DEFAULT 'active',
    created_at  TIMESTAMPTZ DEFAULT now(),
    updated_at  TIMESTAMPTZ DEFAULT now()
);

```

-- 用户↔组织↔角色 关联（一个用户可属于多品牌多组织）

```

user_org_roles (
    id          UUID PK DEFAULT gen_random_uuid(),
    user_id     UUID NOT NULL REFERENCES users(id),
    org_id      UUID NOT NULL REFERENCES organizations(id),
    role_id     UUID NOT NULL REFERENCES roles(id),
    is_default  BOOLEAN DEFAULT false,
    created_at  TIMESTAMPTZ DEFAULT now(),
    UNIQUE(user_id, org_id, role_id)
);

```

4.2 商品模型 (SPU/SKU + 价格策略)

```
-- 商品分类（树形，PostgreSQL LTREE 物化路径）
categories (
  id          UUID PK DEFAULT gen_random_uuid(),
  brand_id    UUID NOT NULL,
  parent_id   UUID REFERENCES categories(id),
  name        VARCHAR(100),
  sort_order  INT DEFAULT 0,
  path        LTREE,          -- 物化路径加速子树查询
  created_at  TIMESTAMPTZ DEFAULT now()
);

-- SPU 标准商品单元（编码一旦生成不可修改）
spus (
  id          UUID PK DEFAULT gen_random_uuid(),
  brand_id    UUID NOT NULL,
  category_id UUID REFERENCES categories(id),
  spu_code    VARCHAR(50) UNIQUE NOT NULL,
  name        VARCHAR(200),
  unit        VARCHAR(20),     -- 计量单位
  spec        VARCHAR(200),
  images      JSONB,
  shelf_life_days INT,
  storage_type ENUM('ambient','refrigerated','frozen'),
  is_active   BOOLEAN DEFAULT true,
  created_at  TIMESTAMPTZ DEFAULT now(),
  updated_at  TIMESTAMPTZ DEFAULT now()
);

-- SKU 最小销售单元（编码一旦生成不可修改）
skus (
  id          UUID PK DEFAULT gen_random_uuid(),
  spu_id      UUID NOT NULL REFERENCES spus(id),
  brand_id    UUID NOT NULL,
  sku_code    VARCHAR(50) UNIQUE NOT NULL,
  spec_detail VARCHAR(200),    -- 规格：500g/袋×20袋/箱
  price       INTEGER NOT NULL, -- 基准售价（分），服务端定价
  cost_price  INTEGER,         -- 成本价（分），仅总部可见
  weight_kg   DECIMAL(8,3),
  min_order_qty INT DEFAULT 1, -- 最小起订量
```

```

    step_order_qty INT DEFAULT 1,      -- 订货步长（只能整箱订）
    is_active      BOOLEAN DEFAULT true,
    created_at     TIMESTAMPTZ DEFAULT now(),
    updated_at     TIMESTAMPTZ DEFAULT now()
);

-- 价格策略（防加盟店改价的核心防线）
-- 优先级：合同价 > 活动价 > 等级价 > 基准价
price_policies (
    id            UUID PK DEFAULT gen_random_uuid(),
    brand_id      UUID NOT NULL,
    sku_id        UUID NOT NULL REFERENCES skus(id),
    policy_type    ENUM('default','store_level','promotion','contract'),
    target_id     UUID,                -- 等级ID/合同ID
    price         INTEGER NOT NULL,    -- 价格（分）
    start_at      TIMESTAMPTZ,
    end_at        TIMESTAMPTZ,
    created_at    TIMESTAMPTZ DEFAULT now(),
    UNIQUE(sku_id, policy_type, target_id, start_at)
);

```

4.3 订单模型（快照 + 不可删除）

```

-- 订单类型 & 状态枚举
CREATE TYPE order_type_enum AS ENUM('purchase','sale','return','transfer');
CREATE TYPE order_status_enum AS ENUM(
    'draft','pending_approval','approved','rejected',
    'pending_production','in_production','partially_produced','produced',
    'partially_shipped','shipped','received','cancelled'
);

-- 订单头
orders (
    id            UUID PK DEFAULT gen_random_uuid(),
    brand_id      UUID NOT NULL,
    order_no      VARCHAR(32) UNIQUE NOT NULL,    -- 业务单号
    store_id      UUID NOT NULL,                  -- 下单加盟店
    order_type     order_type_enum DEFAULT 'sale',
    order_status   order_status_enum DEFAULT 'draft',
    total_amount   INTEGER NOT NULL,              -- 金额（分）

```

```

payment_method  ENUM('balance','wechat','credit','mixed'),
shipping_address JSONB,
expected_at     DATE,
notes           TEXT,
created_by      UUID NOT NULL,
submitted_at    TIMESTAMPTZ,
approved_at     TIMESTAMPTZ,
produced_at     TIMESTAMPTZ,
shipped_at      TIMESTAMPTZ,
received_at     TIMESTAMPTZ,
cancelled_at    TIMESTAMPTZ,
created_at      TIMESTAMPTZ DEFAULT now(),
updated_at      TIMESTAMPTZ DEFAULT now()
);

CREATE INDEX idx_orders_store_status ON orders(store_id, order_status, created_at DE
SC);

CREATE INDEX idx_orders_brand_status ON orders(brand_id, order_status, created_at DE
SC);

-- 订单明细（快照关键信息、不可删除、只能软取消）
order_items (
  id          UUID PK DEFAULT gen_random_uuid(),
  order_id    UUID NOT NULL REFERENCES orders(id),
  brand_id    UUID NOT NULL,
  sku_id      UUID NOT NULL,
  sku_code    VARCHAR(50) NOT NULL,          -- 快照: SKU编码
  sku_name    VARCHAR(200) NOT NULL,         -- 快照: SKU名称
  unit_price  INTEGER NOT NULL,              -- 快照: 下单时的最终价格（分）
  quantity    DECIMAL(10,3) NOT NULL,
  shipped_qty  DECIMAL(10,3) DEFAULT 0,
  received_qty DECIMAL(10,3) DEFAULT 0,
  amount      INTEGER NOT NULL,              -- unit_price × quantity（分）
  status      ENUM('normal','cancelled','short') DEFAULT 'normal',
  lot_no      VARCHAR(50),                  -- 发货批次号（食品安全追溯）
  created_at  TIMESTAMPTZ DEFAULT now()
);

-- 订单审核记录（Append-Only）
order_approvals (
  id          UUID PK DEFAULT gen_random_uuid(),
  order_id    UUID NOT NULL REFERENCES orders(id),
  brand_id    UUID NOT NULL,

```



```

    approver_id      UUID NOT NULL,
    approval_type    ENUM('submit','review','reject','cancel'),
    comment          TEXT,
    created_at       TIMESTAMPTZ DEFAULT now()
);

-- 订单状态变更日志 (Append-Only)
order_status_logs (
    id               UUID PK DEFAULT gen_random_uuid(),
    order_id         UUID NOT NULL REFERENCES orders(id),
    brand_id         UUID NOT NULL,
    from_status      order_status_enum,
    to_status        order_status_enum NOT NULL,
    operator_id      UUID,
    remark           TEXT,
    created_at       TIMESTAMPTZ DEFAULT now()
);

```

4.4 库存模型 (批次 + 预占分离 + 流水不可改)

```

-- 仓库表
warehouses (
    id               UUID PK DEFAULT gen_random_uuid(),
    brand_id         UUID NOT NULL,
    org_id           UUID NOT NULL,
    warehouse_type   ENUM('finished','raw_material','return','virtual'),
    name             VARCHAR(100),
    address          JSONB,
    created_at       TIMESTAMPTZ DEFAULT now()
);

-- 库存表 (SKU + 仓库 + 批次 维度, FIFO拣货)
inventory (
    id               UUID PK DEFAULT gen_random_uuid(),
    brand_id         UUID NOT NULL,
    warehouse_id     UUID NOT NULL REFERENCES warehouses(id),
    sku_id           UUID NOT NULL REFERENCES skus(id),
    lot_no           VARCHAR(50) NOT NULL,          -- 生产/采购批次号
    quantity         INTEGER NOT NULL DEFAULT 0,    -- 实物库存
    locked_qty       INTEGER NOT NULL DEFAULT 0,    -- 预占库存

```

```

    available_qty    INTEGER GENERATED ALWAYS AS (quantity - locked_qty) STORED,
    unit             VARCHAR(20),
    produced_at      DATE,                                -- 生产日期
    expiry_at        DATE,                                -- 效期 (FIFO拣货)
    status           ENUM('normal','quarantined','scrapped') DEFAULT 'normal',
    updated_at       TIMESTAMPTZ DEFAULT now(),
    UNIQUE(warehouse_id, sku_id, lot_no)
);

CREATE INDEX idx_inventory_available ON inventory(sku_id, warehouse_id, expiry_at, available_qty)
WHERE status = 'normal';

-- 库存流水 (Append-Only, 财务审计底层凭证)
inventory_transactions (
    id                UUID PK DEFAULT gen_random_uuid(),
    brand_id          UUID NOT NULL,
    warehouse_id      UUID NOT NULL,
    sku_id            UUID NOT NULL,
    lot_no            VARCHAR(50) NOT NULL,
    trans_type        ENUM('purchase_in','production_in','return_in','sale_out',
                          'scrap_out','transfer_out','transfer_in',
                          'adjustment','lock','unlock','initial'),
    quantity          INTEGER NOT NULL,                  -- 正=入库, 负=出库
    balance_after     INTEGER NOT NULL,
    biz_type          VARCHAR(50),                       -- 关联业务类型
    biz_id            UUID,
    biz_no            VARCHAR(50),
    operator_id       UUID,
    remark            TEXT,
    created_at        TIMESTAMPTZ DEFAULT now()
);

-- ⚠ 禁止 UPDATE/DELETE 在此表, 应用层强约束

-- 在途库存 (发货后、签收前)
in_transit_inventory (
    id                UUID PK DEFAULT gen_random_uuid(),
    brand_id          UUID NOT NULL,
    shipment_id       UUID NOT NULL,
    order_id          UUID NOT NULL,
    sku_id            UUID NOT NULL,
    lot_no            VARCHAR(50),
    quantity          INTEGER NOT NULL,

```

```

    status          ENUM('in_transit','received','partial_received') DEFAULT 'in_transit',
    shipped_at       TIMESTAMPTZ DEFAULT now(),
    received_at      TIMESTAMPTZ
);

```

4.5 账户模型（金额以分为单位 + 流水不可改）

```

-- 加盟店账户
store_accounts (
    id                UUID PK DEFAULT gen_random_uuid(),
    brand_id          UUID NOT NULL,
    store_id          UUID NOT NULL UNIQUE REFERENCES organizations(id),
    balance           INTEGER NOT NULL DEFAULT 0,           -- 余额（分）
    credit_limit       INTEGER DEFAULT 0,                  -- 信用额度（分）
    credit_days        INT DEFAULT 30,                     -- 账期天数
    frozen_amount      INTEGER DEFAULT 0,                  -- 冻结金额（分）
    available_balance INTEGER GENERATED ALWAYS AS
        (balance - frozen_amount) STORED,
    status            ENUM('normal','frozen','closed') DEFAULT 'normal',
    updated_at        TIMESTAMPTZ DEFAULT now()
);

-- 账户流水（Append-Only）
account_transactions (
    id                UUID PK DEFAULT gen_random_uuid(),
    brand_id          UUID NOT NULL,
    store_id          UUID NOT NULL,
    order_id          UUID,
    trans_type        ENUM('recharge','order_pay','refund','adjustment','credit_repay'),
    amount            INTEGER NOT NULL,                    -- 金额（分）
    balance_after      INTEGER NOT NULL,                   -- 变动后余额（分）
    biz_no            VARCHAR(50),                         -- 微信支付流水号
    remark            TEXT,
    created_at        TIMESTAMPTZ DEFAULT now()
);

CREATE INDEX idx_acct_trans_store_time ON account_transactions(store_id, created_at);

-- 应收账款（账期订单欠款）

```

```

receivables (
  id          UUID PK DEFAULT gen_random_uuid(),
  brand_id    UUID NOT NULL,
  store_id    UUID NOT NULL,
  order_id    UUID NOT NULL UNIQUE REFERENCES orders(id),
  amount      INTEGER NOT NULL,                -- 金额（分）
  paid_amount INTEGER DEFAULT 0,
  due_date    DATE NOT NULL,
  status      ENUM('pending','partial','paid','overdue','written_off') DEFAULT
'pending',
  settled_at  TIMESTAMPTZ,
  created_at  TIMESTAMPTZ DEFAULT now()
);

```

4.6 RBAC 权限模型

```

roles (
  id          UUID PK DEFAULT gen_random_uuid(),
  brand_id    UUID,                          -- NULL 表示全局角色
  code        VARCHAR(50) NOT NULL,
  name        VARCHAR(100),
  description  TEXT,
  UNIQUE(brand_id, code)
);

permissions (
  id          UUID PK DEFAULT gen_random_uuid(),
  code        VARCHAR(100) UNIQUE NOT NULL,  -- resource:action
  resource    VARCHAR(50),
  action      VARCHAR(50),
  description  TEXT
);

role_permissions (
  role_id      UUID NOT NULL REFERENCES roles(id),
  permission_id UUID NOT NULL REFERENCES permissions(id),
  PRIMARY KEY(role_id, permission_id)
);

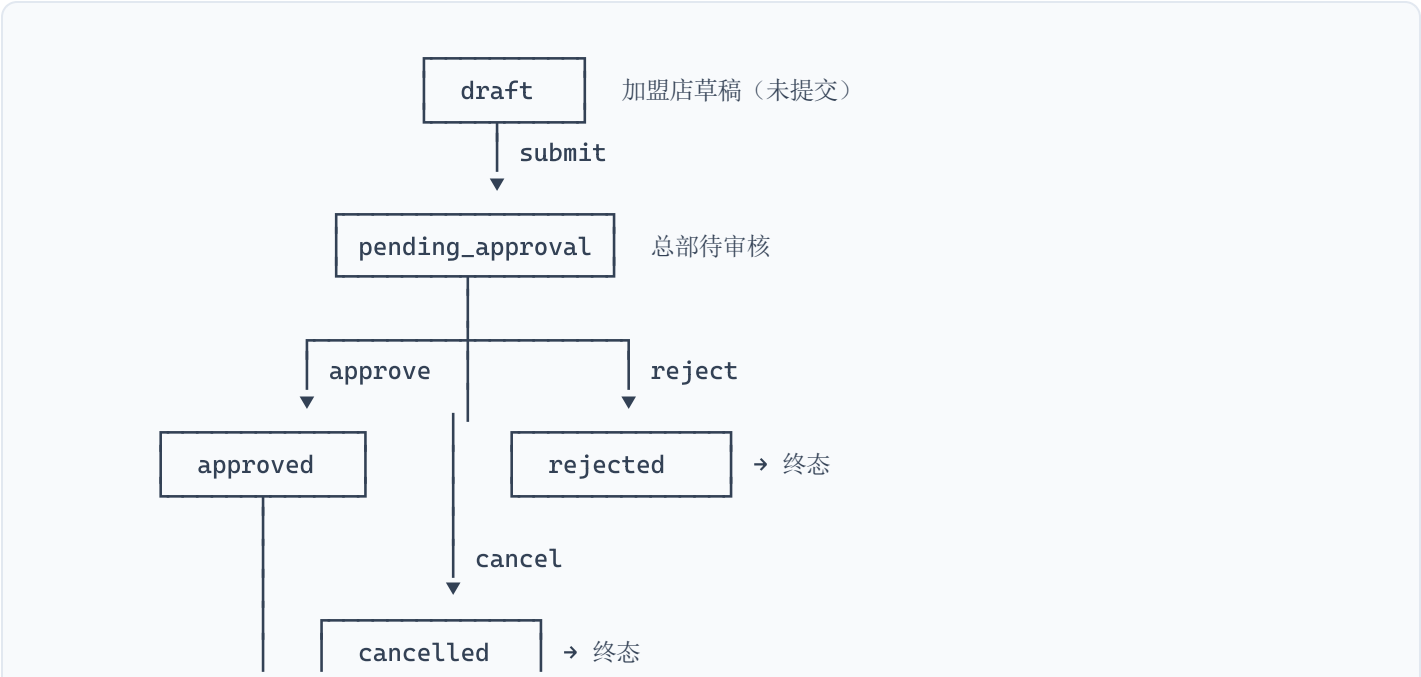
```

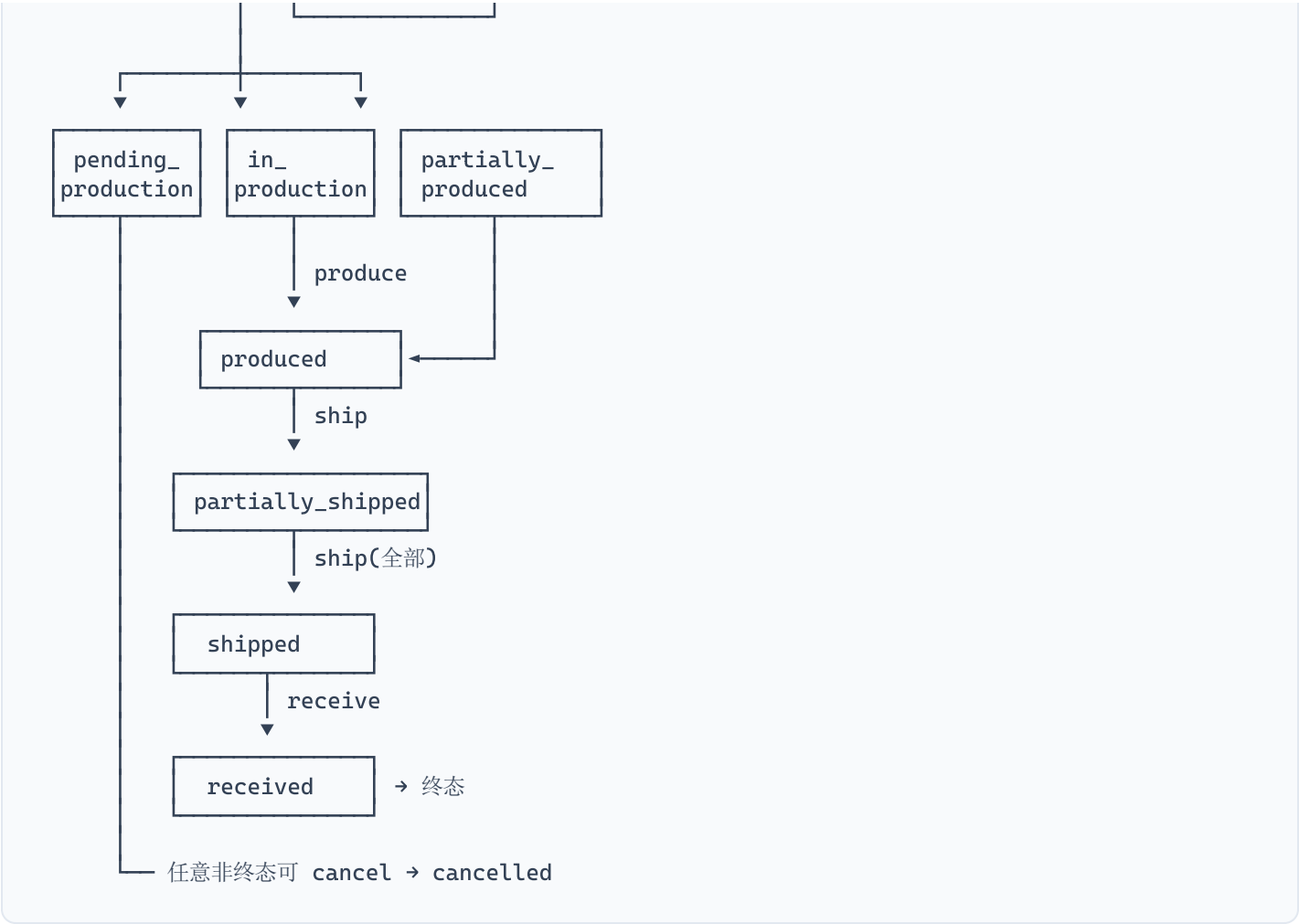
4.7 核心设计原则总结

原则	说明
金额用分存储	所有金额字段使用 INTEGER 存分，杜绝浮点误差，仅展示层转元
价格服务端计算	客户端不传价格，下单时服务端从 price_policies 按优先级取最终价
订单快照	order_items 冗余 sku_code/sku_name/unit_price，SKU 变更不影响历史
流水不可改	inventory_transactions、account_transactions 只能 INSERT
编码不可改	spu_code、sku_code 一旦生成不可修改，所有单据引用编码
批次贯穿全链	lot_no 贯穿入库→库存→出库→订单，确保食品安全可追溯
预占与实扣分离	locked_qty 和 quantity 分开管理，available_qty 为计算列

5. 订单状态机

5.1 状态流转图





5.2 状态与库存/资金联动

状态变更	库存动作	资金动作
submit → pending_approval	无	无
approve → approved	预占库存 lock	余额：冻结金额；账期：扣减信用额度
reject → rejected	无（从未锁定）	无
approved → cancelled	释放预占 unlock	解冻余额 / 恢复信用额度
approved → in_production	预占保持	资金冻结保持

in_production → produced	原料出库 <code>production_out</code> / 成品入库 <code>production_in</code>	资金冻结保持
produced → shipped	预占转实扣 <code>unlock</code> + <code>sale_out</code>	冻结转实扣（余额）或 生成应收（账期）
shipped → received	无	无（账期则应收已生成）

5.3 状态机实现核心

```
// 状态转移规则定义
interface StateTransition {
  from: OrderStatus | OrderStatus[];
  to: OrderStatus;
  allowedRoles: string[];      // 谁有权执行此操作
  preConditions: string[];     // 前置条件检查函数名
  sideEffects: string[];      // 副作用函数名（发MQ、锁库存等）
}

// 核心转移方法
async transition(orderId: string, to: OrderStatus, ctx: TransitionContext) {
  // 1. 校验状态转移合法性
  // 2. 执行 preConditions（余额是否够、库存是否够）
  // 3. 校验操作者角色权限
  // 4. 乐观锁更新: UPDATE WHERE status = currentStatus
  // 5. 记录 order_status_logs（Append-Only）
  // 6. 发布 OrderStatusChangedEvent 到 RabbitMQ
  // 7. 异步消费者处理 sideEffects
}
```

5.4 超时自动取消

审核通过后 N 小时未排产 → 延迟队列消息 → 检查订单仍为 approved →
→ 自动 cancel → 释放预占库存 + 解冻余额 + 推送通知

6. API 模块拆分

6.1 NestJS Module 结构

```
src/
├── app.module.ts
├── common/
│   ├── decorators/      # @CurrentUser, @BrandContext, @Public, @RequirePermission
│   ├── guards/          # JwtAuthGuard, RbacGuard, BrandGuard
│   ├── interceptors/    # BrandContextInterceptor, LoggingInterceptor
│   ├── filters/         # GlobalExceptionHandler
│   ├── pipes/           # ValidationPipe
│   └── dto/             # 通用 DTO
├── modules/
│   ├── auth/            # 认证模块
│   │   └── strategies/  # jwt.strategy.ts, wechat.strategy.ts
│   ├── rbac/            # 角色权限模块
│   ├── brand/           # 品牌管理
│   ├── organization/    # 组织架构 + 加盟申请审核
│   ├── product/         # 商品中心(分类/SPU/SKU/价格策略)
│   ├── order/           # 订单中心(下单/审核/状态机)
│   │   ├── order.state-machine.ts
│   │   └── order.listener.ts
│   ├── inventory/       # 库存中心(锁/释放/扣减/入库/出库/盘点)
│   │   └── inventory.listener.ts
│   ├── procurement/     # 采购管理(总部→供应商)
│   ├── production/      # 生产管理(CK BOM/工单)
│   ├── logistics/       # 物流发货
│   ├── finance/         # 财务中心(账户/应收/对账/账单)
│   ├── payment/         # 支付模块(微信支付V3/回调/幂等)
│   │   └── payment.listener.ts
│   ├── supplier/        # 供应商门户
│   ├── notification/    # 消息通知(微信模板消息/短信/站内信)
│   │   └── notification.listener.ts
│   ├── report/          # 报表中心
│   └── file/            # 文件管理(MinIO/OSS)
```

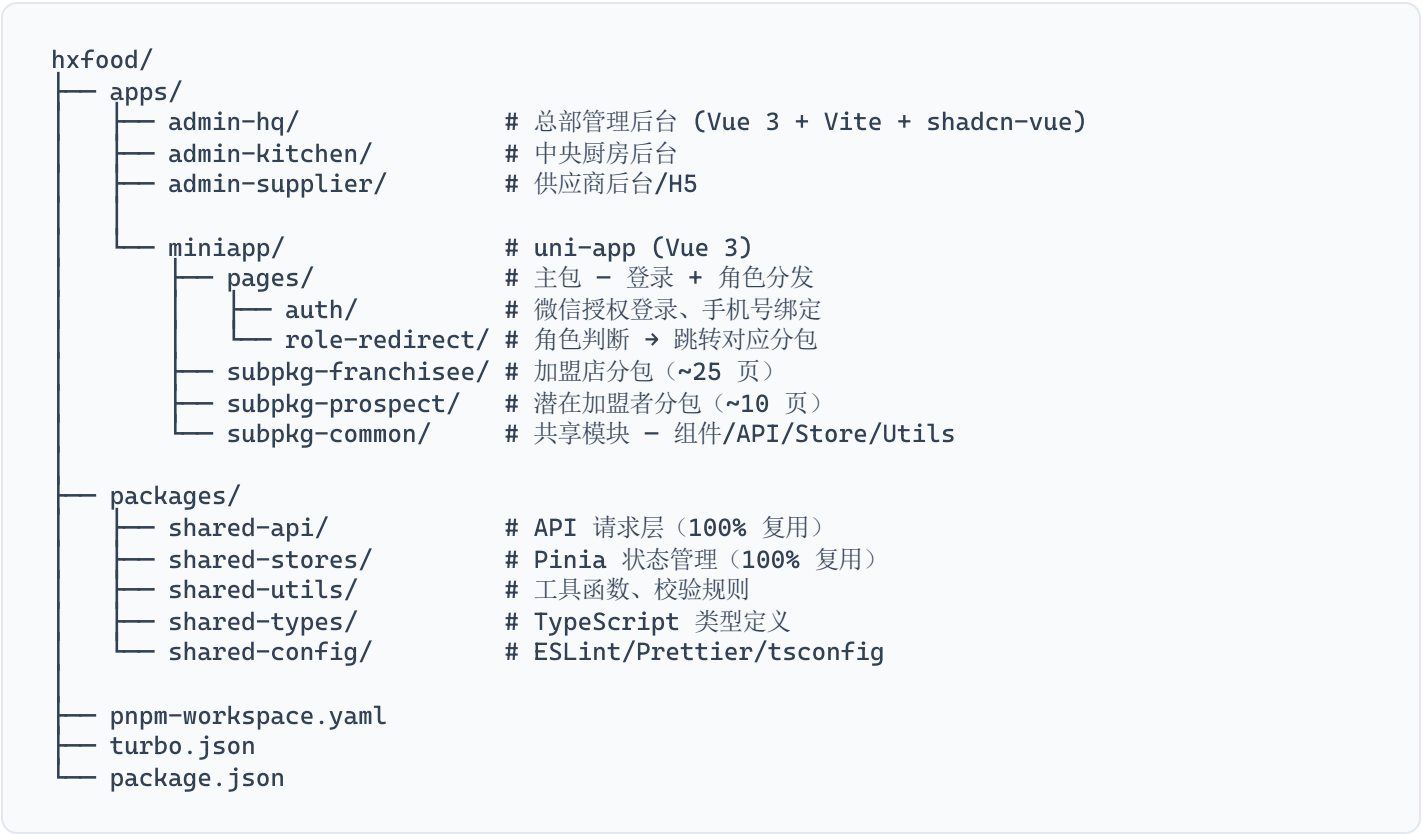
6.2 核心接口清单

模块	接口	方法	调用方
auth	/auth/wechat-login	POST	小程序
auth	/auth/login	POST	Web后台
auth	/auth/refresh	POST	全端

product	/skus	GET	加盟店（含价格）
product	/skus/:id	GET	加盟店（详情+库存）
order	/orders	POST	加盟店下单
order	/orders	GET	按状态/时间查询
order	/orders/:id	GET	订单详情
order	/orders/:id/approve	POST	总部审核通过
order	/orders/:id/reject	POST	总部驳回
order	/orders/:id/cancel	POST	取消
order	/orders/:id/receive	POST	加盟店确认收货
inventory	/inventory	GET	库存查询
inventory	/inventory/check	POST	盘点
production	/production-orders	POST	CK建生产工单
logistics	/shipments	POST	发货
procurement	/purchase-orders	POST	总部采购
finance	/accounts/:id/recharge	POST	加盟店充值
finance	/reconciliation/generate	POST	生成对账单
finance	/receivables	GET	应收查询
payment	/payment/callback/wechat	POST	微信支付回调
report	/reports/sales	GET	销售报表
organization	/organizations/:id/franchise-apply	POST	提交加盟申请
organization	/franchise-applications/:id/approve	POST	总部审批加盟

7. 前端架构

7.1 项目结构 (pnpm monorepo)



7.2 小程序分包策略

同一小程序、独立分包，加盟店端和加盟者端隔离：

分包	内容	访问权限
主包	登录 + 角色分发	所有人
subpkg-franchisee	加盟店端	已授权加盟商
subpkg-prospect	加盟者端	游客可浏览品牌，申请需登录

7.3 加盟店端核心页面

工作台 → 今日订单数、待付款、待发货、公告

商品中心 → 分类/列表/详情/搜索（含价格和库存）

订货 → 选商品+数量 → 确认订单 → 选择支付方式 → 提交

订单 → 列表（按状态筛选）+ 详情（商品/金额/物流/时间线）

账户 → 余额/账期额度/充值/账单列表/还款

门店 → 信息/资质上传

消息 → 通知列表

报表 → 销售看板/热销排行

7.4 加盟者端核心页面

品牌 → 列表/详情/门店实景

加盟申请 → 指南/填表(OCR营业执照)/上传资料/电子签名

进度 → 时间轴跟踪

工具 → ROI测算器

7.5 组件复用策略

模块	WEB	小程序	复用方式
API 请求层	直接引用	直接引用	✔ 100% 复用
Pinia Store	直接引用	直接引用	✔ 100% 复用
工具函数/校验	直接引用	直接引用	✔ 100% 复用
类型定义	直接引用	直接引用	✔ 100% 复用
UI 组件	shadcn-vue	uni-app 原生	✘ 不复用，各自实现

7.6 弱网与离线策略

- 商品数据: 本地缓存 + 后台静默增量更新（lastModified 比对）
- 价格和库存: 必须实时获取，不缓存

- **购物车:** 纯本地存储 (Pinia + uni.setStorage)
- **下单:** 本地草稿 + 自动重试 (最多3次) + clientId 防重复
- **支付:** 必须在线, 支付超时主动查询后端确认状态

8. 安全设计

8.1 价格防篡改 (最高优先级)

```
// 下单核心逻辑 - 价格必须服务端权威计算
async createOrder(dto: CreateOrderDto, brandId: string, storeId: string) {
  return await this.prisma.$transaction(async (tx) => {
    for (const item of dto.items) {
      // 1. 从数据库获取 SKU
      const sku = await tx.sku.findUnique({ where: { id: item.skuId, brandId } });

      // 2. 查加盟店适用价格 (不信任客户端传入价格)
      const price = await this.priceService.getEffectivePrice(
        item.skuId, storeId, brandId
      );

      // 3. 强制用服务端价格, 忽略客户端 price
      orderItems.push({
        skuId: sku.id,
        skuCode: sku.skuCode,    // 快照
        skuName: sku.name,      // 快照
        unitPrice: price,        // ← 服务端定价
        quantity: item.quantity,
        amount: price * item.quantity, // 整数运算, 单位分
      });
    }
    // ...
  });
}
```

8.2 鉴权体系

JWT 双 Token:

access_token → 2小时有效期, Redis 黑名单支持主动吊销

refresh_token → 7天有效期, 绑定设备指纹

JWT Payload:

sub, orgId, orgType, brands[], roles[], permissions[]

RBAC 权限码 (resource:action):

总部超管: **

总部运营: order:approve, product:*, store:view, report:*

总部财务: finance:*, order:view, report:finance

加盟店长: order:create, order:view, order:cancel, product:view, account:view

CK管理员: production:*, inventory:*, shipment:create

供应商: purchase_order:view, shipment:create, reconciliation:view

8.3 其他安全措施

措施	说明
幂等键	下单接口要求 <code>idempotency_key</code> , 防重复提交
充值防重	微信支付 <code>transaction_id</code> 唯一索引防重回调
金额整数运算	全链路以分处理, JavaScript 无浮点隐患
流水不可改	库存流水和账户流水只 INSERT 不 UPDATE/DELETE
品牌隔离	三层防护: Interceptor → Prisma Extension → PostgreSQL RLS
小程序安全	code 仅用一次, unionid/openid 服务端换取, 不信任客户端传的用户信息

9. 消息队列设计

9.1 拓扑结构

```
Exchange: hxfood.order.events (topic)
  Queues:
    └─ inventory.order-status      → 库存锁定/释放/扣减
    └─ notification.order-status   → 微信模板消息推送
    └─ finance.order-status        → 应收生成/核销/冻结解冻
    └─ report.order-status         → Redis 报表数据预热

Exchange: hxfood.scheduled.tasks (direct + 延迟队列)
  Queue:
    └─ order-timeout.dlq           → 超时自动取消 + 释放库存

Exchange: hxfood.payment.events (topic)
  Queue:
    └─ finance.payment             → 充值到账 / 幂等处理

Exchange: hxfood.inventory.events (topic)
  Queue:
    └─ notification.low-stock      → 库存预警通知

Cron: reconciliation.monthly       → 定时生成对账单
```

9.2 异步任务清单

操作	异步原因	消费者
订单状态变更	解耦，保证订单状态先落库	inventory, notification, finance
订单超时取消	预占库存需自动释放	order-timeout.handler
库存预警	不影响正常出入库	notification.listener
微信支付回调	幂等处理，避免重复回调	finance.listener
对账结算	定时批量任务	reconciliation.handler
消息推送	IO 密集，不影响主流程	notification.listener
报表缓存	将复杂查询结果预热到 Redis	report.listener

10. 微信生态集成

优先级	能力	应用场景	接入方式
P0	微信支付 V3 JSAPI	充值/支付/退款	<code>wechatpay-node-v3</code> SDK
P0	手机号快速获取	登录/加盟申请	<code><button open-type="getPhoneNumber"></code> + 后端解密
P0	订阅消息	订单状态变更通知	<code>wx.requestSubscribeMessage</code>
P0	公众号模板消息	兜底通知渠道	公众号模板消息 API
P1	OCR 营业执照识别	加盟申请自动填表	<code>wx.ocr.businessLicense</code>
P1	地图选点	门店地址/附近门店	<code>wx.chooseLocation</code> / 腾讯位置服务
P1	小程序客服消息	加盟咨询/订货问题	<code><button open-type="contact"></code>
P1	企业微信互通	招商沟通/总部通知	企业微信 SDK
P2	物流助手	发货轨迹查询	微信物流助手
P2	电子发票	开票	<code>wx.chooseInvoiceTitle</code>
P2	小程序分享	品牌传播/邀请加盟	<code>wx.shareAppMessage</code>

11. 测试策略

11.1 测试金字塔

层级	占比	工具	重点
单元测试	60%	Jest + NestJS Testing	金额计算、库存计算、状态机、权限、DTO 校验
集成测试	25%	Jest + Docker (PG/Redis/RabbitMQ)	Prisma查询、MQ消费、Redis缓存、支付 Mock
E2E测试	15%	Playwright(Web) + minium(小程序)	加盟入驻、订货全流程、充值消费、多角色协作

11.2 核心测试要点

 **订货流程关键用例：**

- 正常: 单商品/多商品/余额支付/微信补差/账期，全链路通过
- 边界: 库存刚好够/差1件/0库存、余额差1分、最小/最大起订量
- 异常: 非法的状态跳转、重复支付、价格变动

 **财务安全测试：**

- 金额精度: 所有计算以分为单位，快照测试覆盖全部金额函数
- 并发: 同时充值+消费、多笔同时消费、退款并发 → 余额终态一致
- 对账: T+1 对账：日终余额 = 期初 + 充值 - 消费 - 退款

11.3 上线决策矩阵

一个模块可以上线，必须同时满足：

条件	标准
功能验收	所有验收条件勾选完成
测试通过率	核心用例 100%，非核心 ≥ 98%

性能达标	P99 < 2s, 错误率 < 0.1%
安全审查	支付相关接口通过安全审查
财务对账	连续 3 天对账无差异
代码覆盖率	新增代码行覆盖率 ≥ 80%
冒烟测试	生产环境冒烟测试通过
回滚方案	可执行回滚方案已预演

12. 扩展性设计

12.1 当前量级评估

- 日均 1000 单 ≈ 峰值 QPS 10-20（集中在 9:00-11:00）
- 单机 PostgreSQL + NestJS 完全满足
- **不需要微服务拆分**，单体 + Module 隔离即可

12.2 扩展预留

维度	当前方案	升级路径
数据库	单机 PG	读写分离（报表走只读副本）
缓存	Redis 单机	Cluster 模式 / 哨兵
搜索	PG 全文索引	Elasticsearch
应用层	单实例	NestJS 无状态 → 多实例 + Nginx 负载均衡
文件	MinIO 单机	OSS + CDN

分库分表	不实施	单表超 5000 万行再考虑
------	-----	----------------

12.3 关键性能优化点

- 商品列表接口 → Redis 缓存 5 分钟
- 订单列表 → DB 复合索引覆盖查询
- 库存查询 → Redis 缓存有库存 SKU 布尔过滤器
- 报表查询 → 走只读副本，结果缓存在 Redis

13. 项目优先级与里程碑

13.1 开发阶段规划

阶段	内容	优先级
Phase 1: 基础设施	monorepo搭建、Prisma Schema、认证模块、RBAC、品牌隔离中间件	P0
Phase 2: 商品+订货	商品中心、订货流程、微信支付集成、订单状态机	P0
Phase 3: 加盟+小程序	组织架构、加盟申请流程、小程序分包搭建、核心页面	P0
Phase 4: 库存+生产	库存中心、仓库管理、中央厨房生产管理、FIFO拣货	P1
Phase 5: 采购+财务	供应商采购、账户管理、对账结算、账期信用	P1
Phase 6: Web后台	总部后台、中央厨房后台、供应商后台（基础框架+核心页面）	P1
Phase 7: 报表+增值	数据看板、物流轨迹、电子发票、离线缓存	P2

13.2 里程碑

里程碑	交付物	验收标准
M1 — 基础跑通	认证+RBAC+品牌隔离	多品牌用户登录、权限校验、数据隔离验证通过
M2 — 订货闭环	商品+下单+支付+审核	加盟店小程序下单→审核→支付 全链路通过
M3 — 加盟闭环	加盟申请+审核+开通	申请→审核→缴费→开通账号 E2E 通过
M4 — 供应链闭环	库存+生产+发货+收货	订单→生产→发货→收货 全链路通过
M5 — 财务闭环	充值+扣款+对账+还款	3天对账无差异，并发测试通过
M6 — 全系统上线	全部模块	上线决策矩阵全部通过

📁 文档路径: docs/superpowers/specs/2026-07-12-chain-restaurant-management-design.md

🔄 下一步: 进入实现计划阶段 (invoke writing-plans skill)